Progress Report

Thierry Sae-Lim – Norikazu Takahashi
Okayama University
June 23, 2026

Research Objective and Goal

The final goal of the project is to implement the distributed algorithm for PCA (Principal Component Analysis) developped by our lab and test its validity based on numerical experiment.

Short-Term Goals

June 23

1. Reading the paper entitled “Fast linear iterations for distributed averaging”, by Lin Xiao and Stephen Boyd.
2. Writing a Python program to simulate the algorithm (the distributed linear iteration) in the paper (Section 1).

June 30

3. Reading the paper entitled “A Novel Iterative Method for Principal Component Analysis”, by Qiuyun Cao, Kotaro Hattori, Tsuyoshi Migita and Norikazu Takahashi.
4. Implementing the Power Method from the paper in Section 2. (Optionnal: Implementing the Update Rule in Section 3)

July 10

5. Change and separating the plot of the classical power method.
6. Understand the distributed algorithm for PCA.
7. Implementing the distributed algorithm for PCA.

July 17

8.

Progress

1. I read the paper, but there are some points I don’t understand: the proofs of theorem 1, the section 4.1 on the constant edge weghts, the section 6 on the computational methods, and the section 7 on the extensions.
2. I finished writing the program.
   (ア) First, I defined a simple adjacency matrix that represents an undirected graph and the

initial values of each nodes x(0) to test with. I implemented a function to initalize the problem: it creates a dictionnary to store data from the adjacency matrix and the chosen initial values. In the dictionnary is stored the nodes indices (that start from 1 and not 0) as keys, and a sub-dictionnary as values. In the sub-dictionnary, given a node selected on the primary disctionnary, is stored the data of the node's neighbors (key: "n"), the degree of the node (key: "d"), and all the taken values of the node (key: "x").

(イ) Next, I wrote a function that creates the weight matrix W from an instance of the problem based on the local-degrees weights (Section 4.2 from the paper).

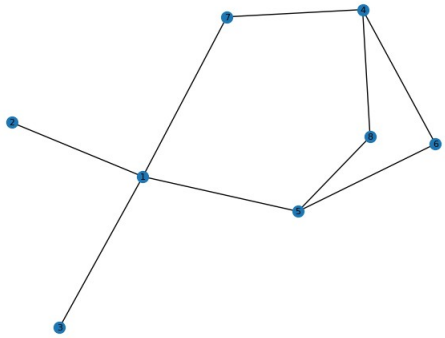
(ウ) And finally, I implemented the distributed linear iteration:

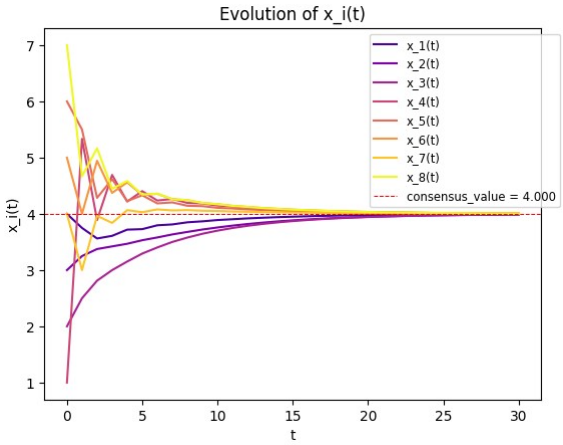
We first need to verify the convergence conditions in the paper (Section 2).

Then we can compute and add the nexts value of each node to the data.

(エ) To manipulate the data more easily, I created 2 getter functions to obtain directly the consensus value wanted, the vector x at time t.

(オ) Testing for the manually defined instance of the problem (N=8, M=9, T=30):





(カ) Everything is working on the example adjacency matrix and the initial values I manually defined previously, but I need to test if it also work on other graphs. So I implemented a function (inspired from the generation method from the paper in Section 5.1) that randomly creates a undirected graph of N nodes and M edges:

Generating the N positions of each nodes and creating the associated complete graph. Next, finding a minimum spanning tree of the graph to ensure its connexity. Then, adding edges one by one by choosing the shortest edge that isn't in the graph yet first till the graph has M edges. Converting the generated graph to its adjacency matrix to use it next. Finally, generating randomly the vector x(0) of the N initial values.

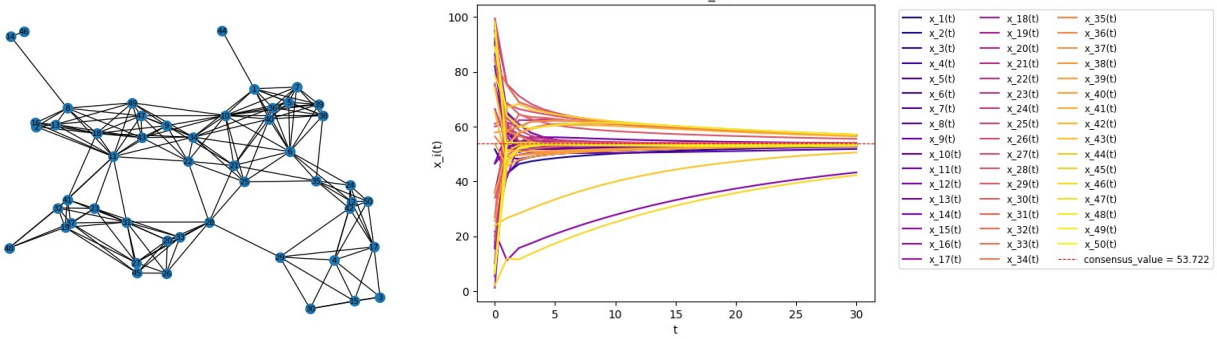
(キ) I created a function to plot the evolution of the values taken by the nodes. The plot is different depending on the parameter i:

- If i isn't set, the evolution of every agents will be shown in a single plot, it should be useful to visualize the global evolution of x over time t.
- if i is set to 0, the evolution of every agents will be shown separately.
- if i is set as a node ID, then the evolution of this agent only will be shown.

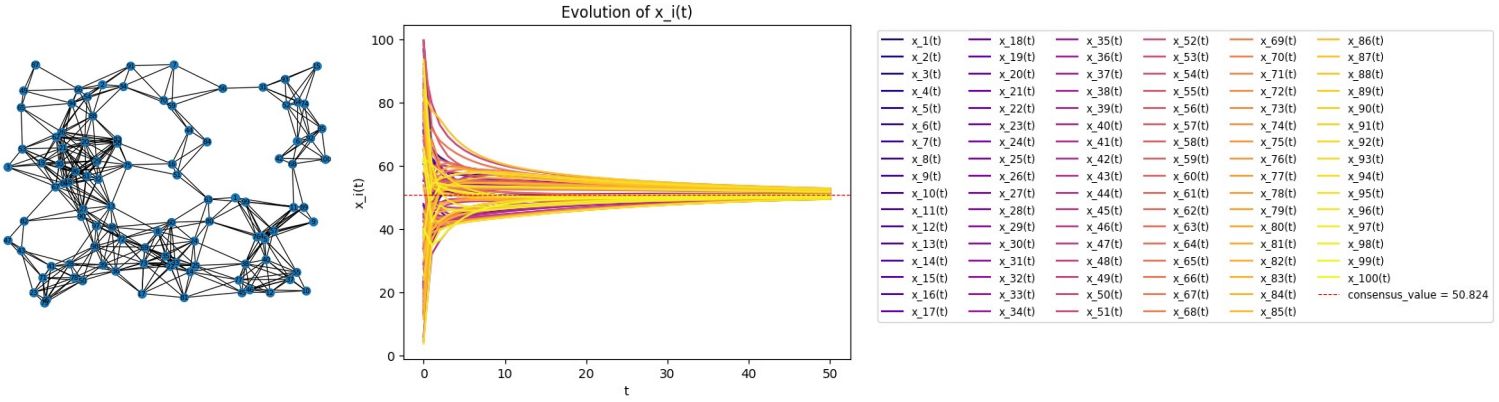
(ク) I also created a fonction to display the graph associated to an adjacency matrix, it should be useful to visualize the randomly generated graphs.

(ケ) Now I can test on randomly generated graphs, by varying the global values of N, M and the number of iteration desired T. Here are some examples:

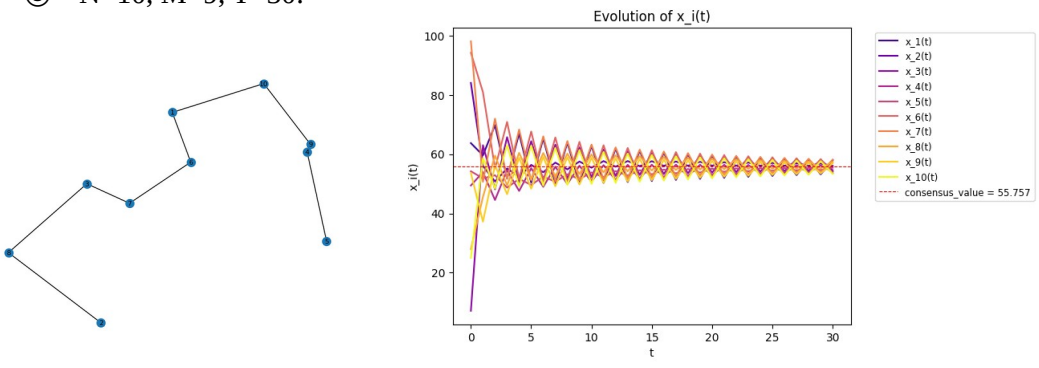
① N=50, M=200, T=30:



② N=100, M=500, T=50:



③ N=10, M=9, T=30:



(コ) The tests are complete and so is the short-term goal 2.

3. I read the paper, but did not read the theoretical analysis section (Section 4).
4. I finished implementing the algorithms in the paper : the classical power method, and the new update rule proposed in the paper.

   (ア) First, to create an instance X of the problem, we can randomly generate NxL matrices and subtract each rows by its average value to ensure that the sum of all entries in each row of X is 0. We then need some initial normalized vectors to start with. We can simply generate random vectors then normlizing them. We next compute the covariance matrix S from X. Here's a quick reminder. We know that S is symmetric and positive semi-definite. Hence, S can be written as S = QΛQ.T with Λ = diag(λ1 ≥ · · · ≥ λN ≥ 0). We want to find the matrix U = (u1 … uP) such as we want to minimize ‖X − U U.T X‖²_F. We also know that the P firsts columns of Q, which we will call Q' = (q1 … qP), is an optimal solution of the PCA optimization problem. So the first step would be to implement functions to find the spectral decomposition of S to find the Q matrix. This can be easily done with the NumPy library.

   (イ) Now that we have an optimal solution, we can implement the classical power method: the data is stored in a Python dictionnary where each key k is associated to a list of all of the estimated k^{th} principal component at time t. So for example, data[3][50] is the estimation of the 3^{rd} principal component at the 50^{th} iteration. So, at each iteration for each principal component, we store the computed value. So the U matrix at the time t is the concatenation of the column vectors of each principal component at the time t: U(t) = (data[1][t] … data[P][t]). Note that each principal component is estimated after the fixed global variable T.

   (ウ) To test the accuracy and the convergence of the power method, we can compute the distance of the vectors u1 to q1 until uP to qP. Note that a principal component is a column of the matrices, that is to say, a direction. So two columns may have the same direction but not the same orientation. In this case, vectors of U would have the same values as vectors of Q', accurate to the sign. So, the computated distances are calculated without taking the orientations of the vectors into account.

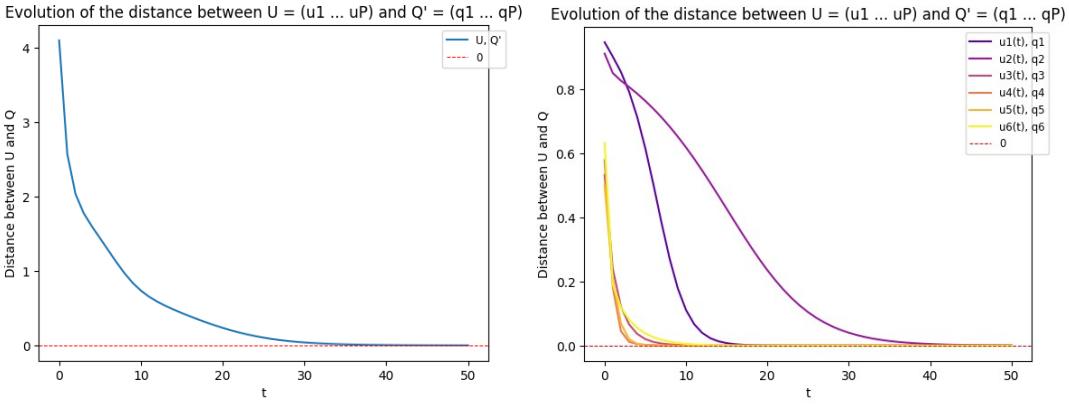
   (エ) I implemented a similar plot function than in the short-term goal 2. The plot function takes an i parameter (default value is None) that determines how to plot:

   - if i=None, the function plots the total distance between U and Q'.
   - if i=0, the function plots the distances between ui(T) and qi in a single plot.
   - if i=-1, the function plots the distances between ui(T) and qi separately.
   - if i is between 1 and P, the function plots the i^{th} principal component only.

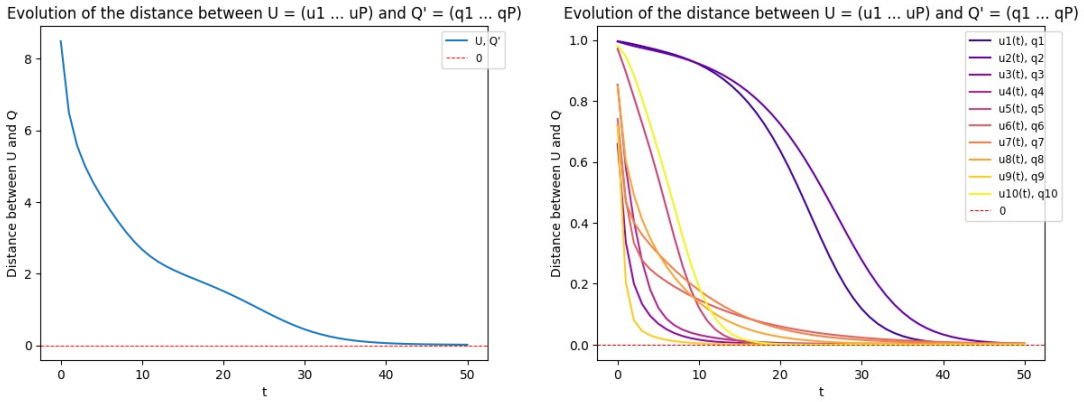
   Here are some example with the evolution of the distances between U and Q' and each

estimated principal component and its optimal value:

① N=10, L=15, P=6, T=50:



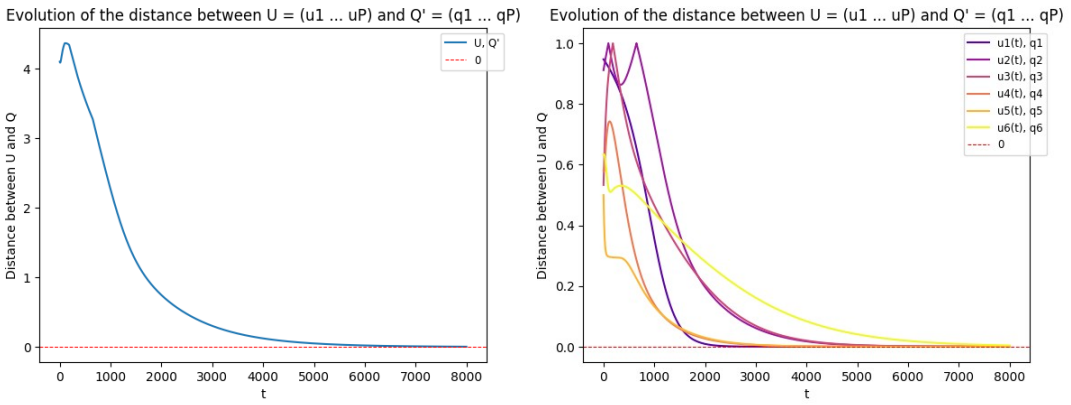
② N= 20, L= 35, P=10, T= 50:



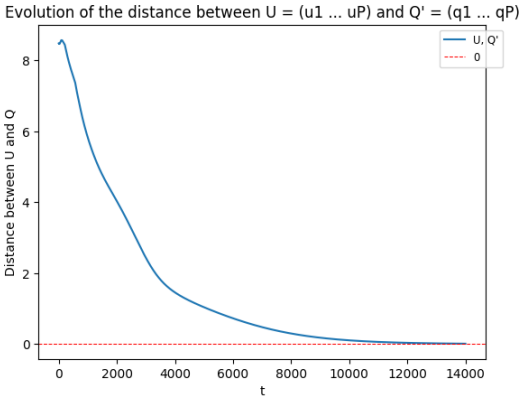
(オ) We can see that the implemented power method is converging to the optimal solution. Now we can implement the update rule proposed in the paper. Note that in the function's loop, we normalize ui(t+1) after computing it, even if the direction is converging to the optimal direction, because we're computing the distance between vectors to verify the correctness of the solution.

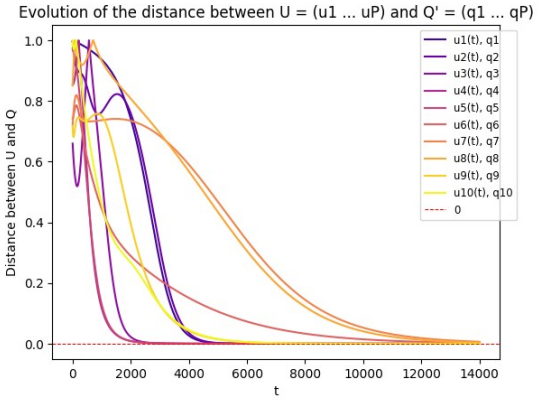
(カ) Here are some plots with the same instances X and initial vectors with fixed update rule parameters, k1=0.2, k2=0.4, epsilon=0.1:

① N=10, L=15, P=6, T=8000:



② N=20, L=35, P=10, T=14000:

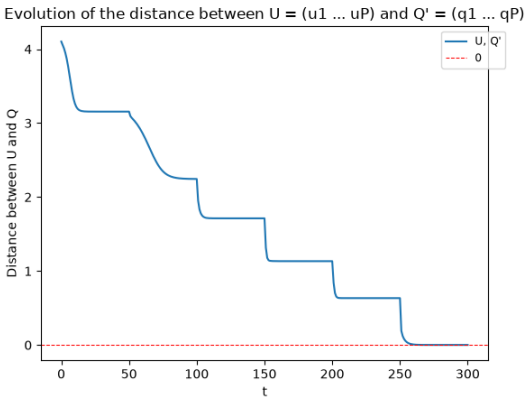


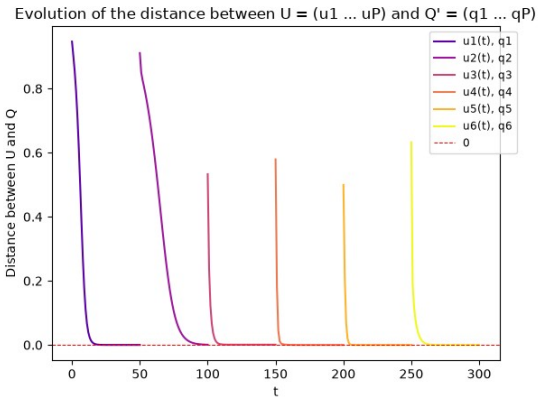


(キ) Although convergence speed isn't the goal of the algorithm, we can observe that the update rule has a much slower convergence speed than the classical power method. A more in-depth study on the update rule parameters' choice (k1, k2 and epsilon) can be conducted. Short-Term goal 4 finish.

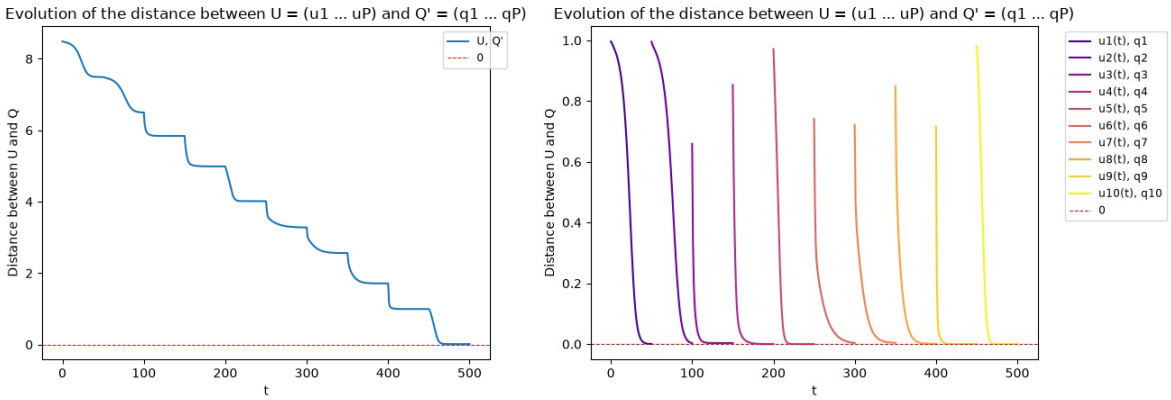
5. Previously, in short-term goal 4 section （エ）, the graphs were incorretly represented. A misunderstanding could arise regarding the classical power method algorithm. It is shown that the principal components are convergening at the same time during each iteration t, but this is incorrect. In the classical power method algorithm, only one principal component is updating during each iteration, and the $i+1^{th}$ principal conponent can only be computed after the i^{th} principal component, more precisely, the i^{th} principal component cannot be updated after the beginning of the $i+1^{th}$ principal conponent's computation. Here is the updated plots one the exact same instances:

(ア) N=10, L=15, P=6, T=50:





(イ) N=20, L=35, P=10, T=50:



6. Here is what I understood from the decentralized algorithm for PCA:

   (ア) We consider an agent network of M agents in which two agents can communicate if and only if ther are neighbors. Each agent m has a private local data matrix X_m of dimension N*L_m (where L_m is the number of sample the agent m has). The goal of the decentralized algorithm for PCA is to find P principal components of the global data matrix X = (X_1 … X_M) of dimension N*L (where L = L_1 + … + L_M) without the agent sharing their private data.

   (イ) To this end, each agent m also has a set P initial vectors of dimension N (u1_m(0) … uP_m(0)) = U_m(0), and then we compute the weight (local-degree weight method) matrix W for the consensus process.

   (ウ) In the first step of the iteration t=0, for each p^{th} principal component, we compute the average consensus of up_m(t) to find yp_m(t). This will results in finding the matrix Y_m(t) = (y1_m(t) … yP_m(t)).

   (エ) The second step of the iteration t=0 consists in computing the matrix Z_m(t) = (z1_m(t) … zP_m(t)), where zp_m(t) is the average consensus of the term X_m*X_m.T*yp_m(t).

   (オ) In the third and last step of the iteration t=0, we compute a single iteration of the update rule (m = 1, …, M ; p = 1, …, P):

   up_m(t+1) = up_m(t) + EPS * (-K1 * sum_j=1_to_p(yj_m(t) * yj_m(t).T * yp_m(t)) + K2 * (M/L_m) * zp_m(t)),

   where EPS, K1 and K2 are the parameters of the update rule.

   (カ) Iteration t=0 done, we can iterate with t ← t+1.

7. I implemented the algorithm cited abouve in the short-term 6.

   (ア) For the initialization, we generate a random agent network, random initial vectors, and random private local data. We compute the weight matrix associated with the network, and we can loop the three steps of decentralized algorithm for PCA.

   The data variable, like the previous implementation contains all the information of the

instance and its computation. First the data has 3 types of keys:

① An integer m refering an Agent's ID which is associated with another sub-dictionary:

1. The string key 'n' associated with the neighbors of agent m.
2. The string key 'd' associated with the degree of agent m.
3. The string key 'X' associated to agent m's private local data matrix X_m.
4. The string key 'U' associated with the history of the computated matrices U_m(t) = (u1_m(t) … uP_m(t)) for each time t, by agent m, in the algorithm's step 3.
5. The string key 'Y' associated with the history of the computated matrices Y_m(t) = (y1_m(t) … yP_m(t)) for each time t, by agent m, in the algorithm's step 1.
6. The string key 'Z' associated with the history of the computated matrices Z_m(t) = (z1_m(t) … zP_m(t)) for each time t, by agent m, in the algorithm's step 2.

Note that the computated matrices U, Y and Z are stored as a list of colum matrices not simple matrices. So for example data[m]['U'][35][3] refers to the 3rd column (and not 3rd row) of the estimated U_m matrix at time t=35 for agent m. In other words, data[m]['U'][35][3] is the 3rd principal component's estimation at time t=35 by agent m. Same structure for the keys 'Y' and 'Z'.

② The string key 'X' associated with the global data matrix X = (X_1 … X_M).

③ The string key 'Q' associated with the global optimal solution of the PCA problem, the matrix of eigenvectors Q = (q1, …, qN) and not Q' = (q1 … qP).

(イ) I implemented a plot function to visualize the distance (accurate to the sign, as explained in short-term goal 4, section （ウ）) of the principal components' estimations to the optimal principal components (eigenvectors of the spectral decomposition of X). This function has two optional parameters to specify what to plot. The first optional parameter m (default value = None) has 3 different behaviors:

- m = None: the plot displays global information (a single curve that is the sum each agent's curve)

- m = 0: Tthe plot displays local information for each agent (one curve associated to one agent)

- 1 <= m <= M: the plot only displays local information associated to the agent m (the curve associated to agent m).

The second optional parameter p (default value = None) has 3 differents behaviors:

- p = None: the plot displays information on the total distance of all P principal

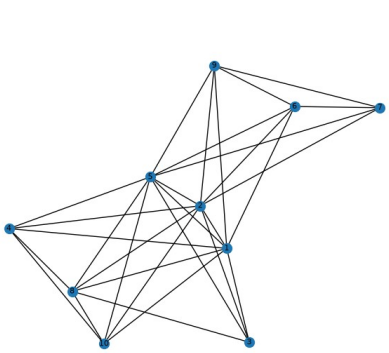
components (a single curve that is the sum of each principal component's curve)

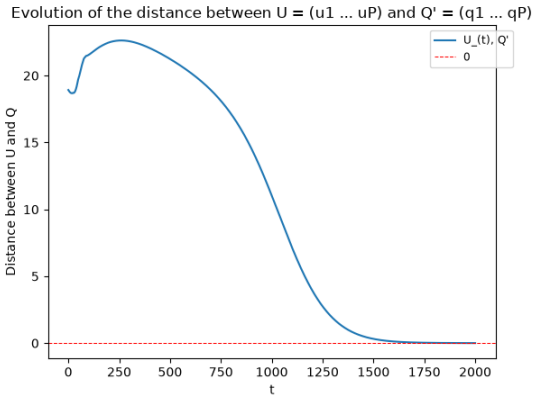
- p = 0: the plot displays information on the distances of all P principal components (one curve associated to one principal component)

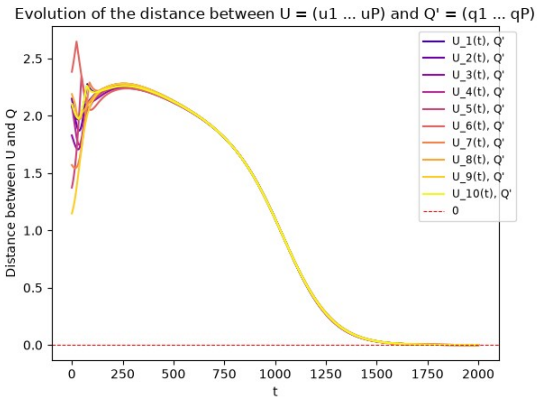
- 1 <= p <= P: the plot only display information associated to the p^{th} principal component (the curve associated to the p^{th} principal component).

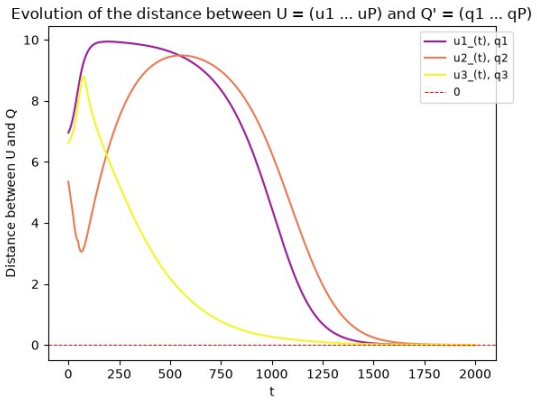
Here are some plots:

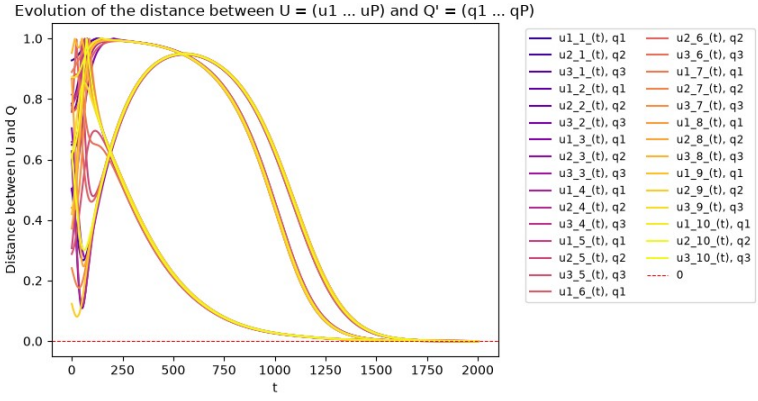
① NB_AGENT=10, NB_EDGES=30, N=DIM=5, P_DIM=3,
K1=0.2, K2=0.4, EPS=0.1
T_CONSENSUS=100, T_POWER_METHOD=1000:



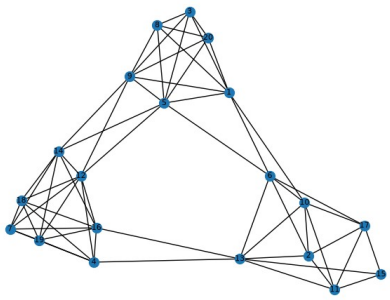


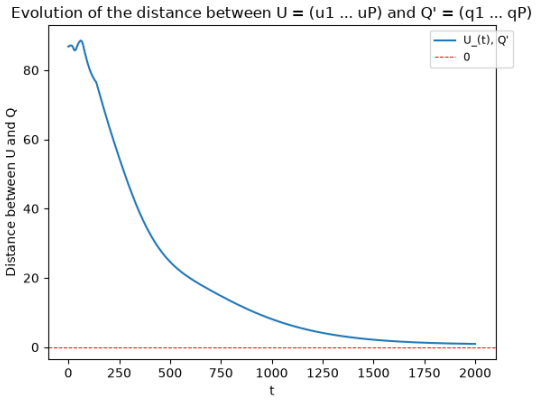


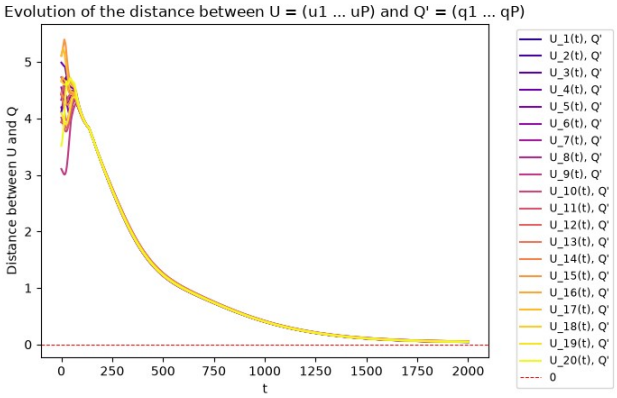


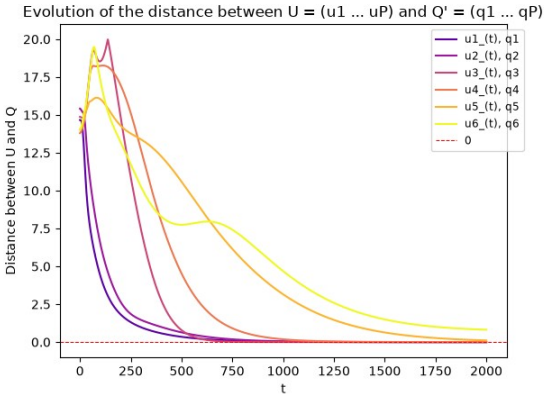


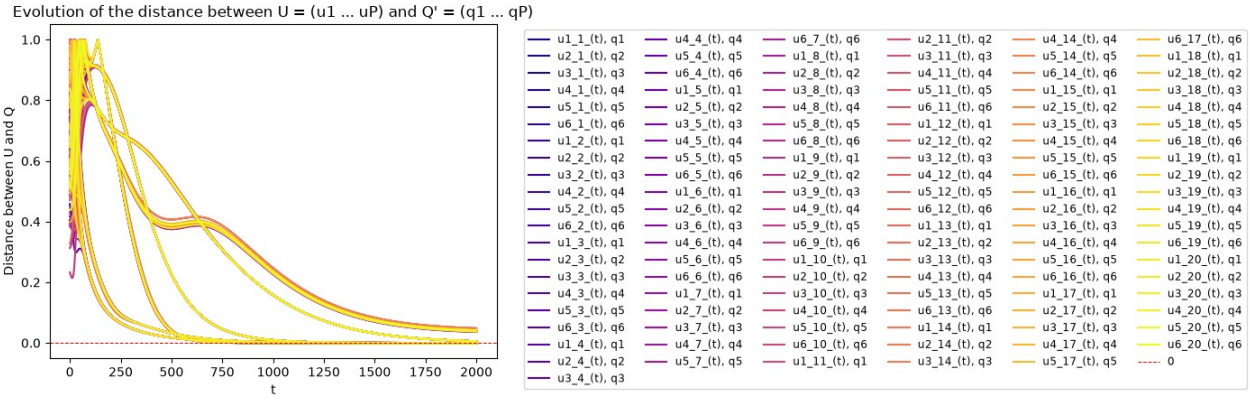
② NB_AGENT=20, NB_EDGES=60, N=DIM=10, P_DIM=6,
K1=0.2, K2=0.4, EPS=0.1
T_CONSENSUS=100, T_POWER_METHOD=2000:











（ウ）The tests are complete and so is the Short-Term goal 7.